USER GUIDE EN — Audion Hub Manager

This user guide covers launch, setup, MIRROR, Git/Auth, Editor/Diff and project recovery.

1. Start

From the project root:

```
launcher_gui.cmd
```

CLI smoke:

```
launcher_cli.cmd --mirror-preview demo_local --json
launcher_cli.cmd --mirror-apply demo_local --json
launcher_cli.cmd --mirror-apply demo_local --apply --json
```

If the portable runtime is missing:

```
install\Build_Portable_Env.cmd
```

2. First Setup

Open:

```
config/projects.json
```

For each project configure:

- `source_path` — live project; relative values resolve from the Hub Manager root.
- `projection_path` — Hub Data mirror.

- `docs_path` — optional Markdown/text reading layer.
- `profile` — MIRROR profile from `config/projection_profiles.json`.

Prefer relative paths for portable projects inside the manager tree:

```
{
  "id": "audion_hub_manager",
  "title": "Audion Hub Manager",
  "source_path": ".",
  "projection_path": "S:/Audion/Hub Data/Audion Hub Manager",
  "docs_path": "S:/Audion/Docs/Projects/Audion Hub Manager",
  "profile": "audion_python_project_projection",
  "default_branch": "main"
}
```

Folder pickers and project scanning save relative paths when the selected folder is inside the Hub Manager tree. Absolute paths still work for external Hub Data and Docs locations.

Source must not be changed by MIRROR. Hub Data can be rebuilt.

3. Project Dropdown And Project Scanning

The `Project` dropdown selects one entry from `config/projects.json`. Selecting a project switches the whole bundle:

```
Project/Source -> Hub Data/Mirror -> Docs
```

For a folder with 20 projects, use this workflow:

1. Click `Rebuild dropdown` in `SOURCE ACTIONS` or `Scan projects` in Storage.
2. Choose the parent folder, for example `S:/TOOLS/Apps/`.
3. Hub Manager detects real project roots, including double-nested folders such as `Project/Project`.
4. Detected projects appear in the `Project` dropdown.
5. Switch projects one by one and run MIRROR/commit for the active project.

The `Structure` panel has three tree layer switches:

```
PROJECT  -> source_path
GIT COPY -> projection_path
DOCS     -> docs_path
```

The compact badge next to the `Structure` title shows the selected layer and resolved path. Folder icons pick a new location for that layer, and the clear button removes location overrides. Tree Search only filters files inside the currently selected layer; it does not switch projects.

The scanner does not change Source folders. It only writes missing entries to `config/projects.json`.

If stale seed/demo entries, missing `source_path` values or duplicates remain in the dropdown, click `Support -> Clean projects.json`. The cleaner edits only `config/projects.json`; it does not delete project folders.

The `SOURCE` badge under `SOURCE ACTIONS` updates after `Batch Git status` and shows the registry-wide summary:

```
SOURCE: <projects> PROJECTS / <clean> CLEAN / <dirty> DIRTY / <errors> ERRORS
```

4. MIRROR Preview And Apply

Preview first:

```
launcher_cli.cmd --mirror-preview <project_id> --json
```

Apply deliberately:

```
launcher_cli.cmd --mirror-apply <project_id> --apply --json
```

MIRROR copies only allowed files. Runtime, caches, logs, builds, binary payloads, local overrides and secrets must stay outside Hub.

5. GUI Panes

The left side of the app:

```
Open
  Open Project | Mirror | Docs Folder
  Open in VS Code | Terminal | Git

MIRROR
  Dry-run | Exact mirror | .gitkeep dirs | BLAKE3 compare
  Preview MIRROR | Apply MIRROR | Refresh

SOURCE ACTIONS
  Rebuild dropdown | Clone Source | Batch Preview MIRROR
  Batch Safety Scan | Batch Verify Mirror | Batch Git status
  Combined workspace

PROJECT ACTIONS
  Refresh tree | Load selected to Editor | Open in VS Code
  Copy relative path | Copy full path

Support
  Auth Doctor | BLAKE3 | Verify Mirror | Storage | Safety Scan
  Clean projects.json
```

`SOURCE ACTIONS` operate on the whole project registry. `PROJECT ACTIONS` operate on the current selected tree object.

`Terminal` opens an external terminal in the active Source project. `Git` opens an external terminal in the current Git root and immediately runs `git status --short`, so it is a Git inspection shortcut rather than another folder-open button.

The right side of the app:

```
Quick  | Branch | Editor | Diff    | Storage
Remote | Basket | Reader | History | Details
```

`Quick` — frequent local Git/file commands. The command area at the bottom is a manual command textarea: selecting a cached/pinned command copies it there, and the run button executes that exact text.

Together, the Git panes cover almost the whole everyday lifecycle: repository creation, status, root lookup, history, diff/blame/show, staging, restore, commit, tags, remotes, branch switching/creation, stash, revert, merge, cherry-pick, bundle, clean preview, gc, fsck, config diagnostics, CI/CD observation and clone into Source. Branch switching, integration, stash and branch-danger templates live in

`Branch`, not in `Quick`, so high-risk branch work stays in one workflow window. The remaining gap is intentional: rare destructive workflows stay as visible command templates rather than one-click magic.

The `GIT LOCAL` block is the first stop for everyday repository state: `init` creates a visible `.git`, `status` shows current changes, `root` prints the repository root, `log --oneline` shows compact recent history and `reflog` helps find rollback points. `user config` shows `user.name`/`user.email` with origin in the Auth block inside `Remote`. `config list` prints all Git config values with their source files in Git backup/maintenance. The graph view remains in `History`.

`Basket` — readable commit preparation: type, scope, subject, message, version/tag.

`Branch` — branch status, `branch -vv`, `switch`, `switch -c`, tags, compare branches, `merge --no-ff`, `revert`, `cherry-pick`, stash and abort/rebase templates.

`Remote` — remotes, push/pull/fetch and authentication setup. The remote form builds SSH URLs from platform/login/repository fields, saves enabled remotes to `config/remotes.json`, and keeps recent field values in `config/remote_field_cache.json`. Recent values are applied only by the explicit use button, never automatically over typed input.

Common remote commands are ordered by frequency:

```
push origin | pull --ff-only
fetch --all --prune | remote -v
push all remotes | apply remotes.json
origin push URLs | build URL
save remote
```

`push origin` queues `git push --follow-tags origin <branch>`, so annotated checkpoint tags are published too. `push all remotes` pushes the branch and tags to every configured remote without a shell-specific PowerShell loop.

`Editor` — quick Markdown/text editor:

```
.md
.markdown
.txt
.rst
```

Its toolbar is icon-only with tooltips: load selected, save, paste from Windows clipboard, copy, clear, open current file in VS Code and expand/collapse the pane.

`Diff` — RedLine view of changes.

Auth actions live inside `Remote`: `Check Auth`, paired GitHub/GitLab SSH probes, paired `gh auth login` / `glab auth login`, Windows Credential Manager, GitKraken folder and VS Code. Login opens an external terminal once; Hub Manager does not store tokens or passwords.

`Storage` — Manager/Hub Data/Docs roots, layout checks, Safety Scan output and machine-local `Code.exe` picker/test/save controls. These values belong in `config/apps.local.json`, not in shared project config.

6. BLAKE3 and Verify Mirror

`BLAKE3` is a quick backend diagnostic: it verifies that MIRROR is using real BLAKE3 rather than the SHA-256 fallback.

`Verify Mirror` is a read-only Source ↔ Hub Data verification after mirroring. It builds BLAKE3 manifests in memory, compares `same / changed / missing / extra`, and writes one timestamped JSON report to:

```
logs/<project_id>/
```

Source, Hub Data and Docs do not receive manifest files. Docs/Obsidian/LogSeq/ VS Code docs folders are not hashed separately: they are a reading layer, and the canonical technical copy already lives in Hub Data.

7. Git Workflow

Hub Manager works with normal `.git` directories:

```
Full Project Source/.git
Hub Data/<project>/.git
```

Commits created by Hub Manager must stage only the pathset accepted by the active Hub profile. Do not use broad `git add .` for these checkpoint commits.

Good commit name:

```
docs(hub): projection v0.1.17 - refresh builder menu
```

Version is manual on purpose. Automatic versions can create noise and false importance.

8. Remote And Auth

Hub Manager does not store tokens.

Use:

```
gh auth login
glab auth login
ssh -T git@github.com
ssh -T git@gitlab.com
```

Windows Credential Manager, VS Code and GitKraken can be used for login, credential-helper checks and visual conflict work.

After auth is configured once on a machine, normal sync is:

```
machine A: commit -> tag -> push origin
machine B: fetch --all --prune -> pull --ff-only
```

Use `fetch` to update remote tracking information without changing local files. Use `pull --ff-only` when you deliberately want to advance the current branch.

8.1. Your own Forgejo or Gitea server

The top row of the `Remote` pane is a set of platform buttons: `GitHub`, `GitLab`, `Codeberg`, `Forgejo`, `Gitea`, `Custom host`. The last three also reveal `Server URL` and `SSH port` fields; cloud platforms do not need them. The second row picks `SSH key` or `HTTPS token`, which decides what `build URL` produces.

```
port 22        -> git@host:owner/repo.git
other SSH port -> ssh://git@host:PORT/owner/repo.git
HTTPS          -> https://host/owner/repo.git
```

Known servers live in `config/forgejo_hosts.json`. It never holds tokens: only the address, SSH port, preferred URL type and the login cached after the first successful check.

Connection steps:

1. `check server` asks the instance for its API version, confirming the address before any token is involved.

2. `token page` opens `Settings -> Applications` on your instance. Create a token there with these scopes:

What you do	Scope
"check server"	no token needed
"who am I"	`read:user`
"list repositories"	`read:user`
"create repository"	`write:user`
`git clone` / `git push` over HTTPS	`read:repository` / `write:repository`

   Scopes follow **the route, not the object**. Creating a repository lives at `/api/v1/user/*`, so it needs `write:user`, not `write:repository` — with the latter the server answers `403 ... required scope(s): [write:user]`. Conversely a token holding only `read:user,write:user` passes every button in the pane but fails `git clone` over HTTPS with `remote: Forbidden`: git is judged separately.

   The simplest choice is to grant both the user and repository scopes at once. Forgejo collapses the redundant ones on save: `read:user,write:user,write:repository` is stored as `write:repository,write:user`, because write implies read. All of this was verified on a live Forgejo 15 with one isolated token per operation. Hub Manager reports a missing scope separately — "the token is valid but was created without the scope this call needs" — and names the one the server asks for.

3. Paste it into `Access token`. That is already enough — `who am I`, `list repositories` and `create repository` work straight away with whatever is in the field. `remember token` does something different: it verifies the token against `/api/v1/user` and hands it to your Git credential helper so it survives a restart and so `git` itself can use it. Either way nothing is written into the project files.

   For `git clone` and `git push` over HTTPS, remembering the token is **required**: git reads credentials from the store and cannot see the field.

4. `who am I`, `list repositories` and `use repository` work off the stored token. `use repository` fills login and repository from the selected row and builds the remote URL immediately.
5. `create repository` creates a repository under your own account from the `Repository` field, with visibility set by the `private` / `public` buttons.
6. The crossed-out key icon next to the token field forgets it: the field is cleared and the remembered copy is removed from the credential store. The token stays valid on the server until you revoke it there, on the same token page.

Pushing is authenticated separately and conventionally: either by an SSH key (`ssh -T Forgejo` probes it and honours a non-standard port), or by the same token over HTTPS, which `git push` reads from the credential store on its own.

`Check Auth` reports, per server: whether it is reachable, whether a token is stored, whether that token is still valid, and which credential helpers are configured.

8.2. A large first push through a proxy or tunnel

If the instance is published through a reverse proxy or tunnel that caps request body size, the first push of a repository can answer `413`. Git sends the pack as a single request, so the cap hits the initial history upload rather than the small commits that follow. On Cloudflare Tunnel's free tier that cap is 100 MB, and it cannot be avoided by DNS settings while the tunnel is in use.

The way around it is to push over SSH, bypassing the HTTP proxy. If the SSH port only listens on localhost on the server, forward it for the duration of the push:

```
ssh -N -L 2222:127.0.0.1:2222 user@server
git push ssh://git@127.0.0.1:2222/owner/repo.git
```

Hub Manager builds such URLs itself: pick `SSH key`, set the address and SSH port, and `build URL` produces `ssh://git@host:PORT/owner/repo.git`. Git LFS does not help if it is disabled on the server or routed through the same proxy.

9. VS Code

The `Code.exe` path is machine-specific. Shared config should not store it.

Store local paths in:

```
config/apps.local.json
```

`*.local.json` files must not enter Hub Projection or Git commits.

10. Recovery From Hub Data

If Source was deleted but Hub Data survived:

```
   git -C "<Hub Data>\Audion Hub Manager" log --oneline -n 5
robocopy "<Hub Data>\Audion Hub Manager" "<portable-root>\Audion Hub Manager" /E
/COPY:DAT /DCOPY:DAT /R:1 /W:1 /XJ
```

After recovery, rebuild runtime and PDF/release artifacts if needed.

11. PDF

PDF is not the source of truth. Markdown is.

PDF can be regenerated with the external engine:

```
python "E:\TOOLS\Audion Office OCR AI\system_core\dev_markdown_pdf_engine.py"
```

12. Minimal Checks

```
python -m compileall -q system_core
python -m pytest -q tests
python system_core\ui_nicegui\app.py --smoke
```

Expected baseline: `59 passed`.